WEEK 9 LAB ASSIGNMENT | EXERCISE 4 COMPLETED | CLO1 & CLO4

รายงานผลการทดสอบและความครอบคลุมของโค้ด (Test Coverage Report)

แบบฝึกหัดที่ 4: ตั้งเป้า Coverage และวิเคราะห์ผลลัพธ์ (Coverage Goal Setting & Branch Analysis)

รหัสนิสิต: 67160178

รายวิชา: การพัฒนาเว็บเซอร์วิส (Web Services & API Development)

โปรเจกต์เป้าหมาย: `student-api` (ไฟล์ทดสอบ: `app.js`)

เครื่องมือทดสอบ: Jest 30.5.2 + Supertest 7.3.1

1. ภาพรวมและเป้าหมายการทดสอบ (Executive Summary & Goal Setting)

รายงานฉบับนี้จัดทำขึ้นเพื่อรายงานผลสัมฤทธิ์ของ **แบบฝึกหัดที่ 4 (ตั้งเป้า Coverage และวิเคราะห์ผลลัพธ์)** ตามคู่มือปฏิบัติการสัปดาห์ที่ 9 โดยมีเป้าหมายหลักคือการยกระดับความครอบคลุมของกิ่งเงื่อนไข **(Branch Coverage) ของไฟล์ `app.js` ให้ได้อย่างน้อย 70.00%**

ตัวชี้วัดความครอบคลุม (Coverage Metric)	ก่อนปรับปรุง (Baseline)	เป้าหมาย (Target)	ผลลัพธ์ที่ได้ (Achieved)	สถานะการประเมิน
% Branch Coverage (เป้าหมายแบบฝึกหัดที่ 4)	**41.67%**	**≥ 70.00%**	**73.53%**	✅ **ผ่านเป้าหมาย (+31.86%)**
% Statement Coverage	52.41%	≥ 70.00%	78.43%	✅ **ผ่านเกณฑ์ (+26.02%)**
% Function Coverage	50.00%	≥ 70.00%	80.00%	✅ **ผ่านเกณฑ์ (+30.00%)**
% Line Coverage	54.10%	≥ 70.00%	78.95%	✅ **ผ่านเกณฑ์ (+24.85%)**

สรุปสถิติจำนวน Test Case ที่เพิ่มขึ้น:

- จำนวน Test Cases เดิมก่อนเริ่มแบบฝึกหัด: **6 Cases** (ครอบคลุมเฉพาะ Happy Path พื้นฐาน)
- จำนวน Test Cases ที่พัฒนาเพิ่มเข้าไป: **8 กรณีทดสอบ (8 Test Cases)**
- **รวมเป็น 14 Test Cases** ส่งผลให้อัตรา Branch Coverage ทะยานจาก 41.67% สู่ **73.53%** บรรลุตามข้อกำหนดอย่างสมบูรณ์

2. การวิเคราะห์ Branch ที่ขาดหายไปใน `app.js` (Detailed Branch Coverage Analysis)

จากการวิเคราะห์ Uncovered Branches ในไฟล์ `app.js` พบว่ากิ่งเงื่อนไขที่ขาดหายไปเกิดจากการตรวจสอบข้อมูลนำเข้าที่ไม่ถูกต้อง (Validation Errors), การตรวจสอบ Header และการตอบสนองต่อข้อผิดพลาด (Error Responses) โดยมีรายละเอียดการเพิ่มเทสทั้ง 8 กรณีดังนี้:

กรณีทดสอบที่ 1: ตรวจสอบ Header Content-Type ใน Middleware `requireJson`

- **Branch ที่ครอบคลุม:** บรรทัดที่ 45-54 เงื่อนไข `if (!req.is("application/json"))` ฝั่ง True
- **Branch เดิมที่ขาด:** ไม่มีเทสที่ส่ง Content-Type อื่น โค้ดในบล็อกปฏิเสธและคืน 415 จึงไม่เคยถูกทดสอบ
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ `POST /api/v1/students` พร้อม Header `Content-Type: text/plain` ยืนยันการตอบกลับ HTTP 415 `UNSUPPORTED_MEDIA_TYPE`

กรณีทดสอบที่ 2: ตรวจสอบ Fallback ของ Request Method ใน `requireJson`

- **Branch ที่ครอบคลุม:** บรรทัดที่ 45 เงื่อนไข `methodsWithBody.includes(req.method)` ฝั่ง False
- **Branch เดิมที่ขาด:** การทดสอบว่าเมธอดแบบ GET/DELETE จะไม่ถูกดักตรวจ Content-Type
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ `GET /api/v1/students` โดยไม่ระบุ Content-Type ยืนยันว่าสามารถเรียก `next()` เข้าทำงานได้ปกติ

กรณีทดสอบที่ 3: ค้นหานักศึกษาด้วย ID ที่ไม่มีในระบบ (`GET /api/v1/students/:id`)

- **Branch ที่ครอบคลุม:** บรรทัดที่ 235-245 เงื่อนไข `if (!student)` ฝั่ง True เมื่อค้นหาไม่พบ
- **Branch เดิมที่ขาด:** เดิมทดสอบเฉพาะ ID ที่มีอยู่จริง ทำให้เงื่อนไขแจ้งเตือนไม่พบข้อมูลไม่เคยถูกเรียก
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ `GET /api/v1/students/9999` ยืนยันการตอบกลับ HTTP 404 `STUDENT_NOT_FOUND`

กรณีทดสอบที่ 4: การตรวจสอบข้อมูลบังคับไม่ครบถ้วนใน `POST /api/v1/students`

- **Branch ที่ครอบคลุม:** บรรทัดที่ 258-270 ตรรกะ `if (!name || !major || !email)` ฝั่ง True
- **Branch เดิมที่ขาด:** ทดสอบเฉพาะการส่ง Body ที่สมบูรณ์ ตรรกะตรวจข้อมูลว่างจึงยังไม่ถูกประเมิน
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ `POST` โดยไม่ส่งฟิลด์ `email` ยืนยันการตอบกลับ HTTP 400 `VALIDATION_ERROR`

กรณีทดสอบที่ 5: การตรวจจับอีเมลซ้ำซ้อนในระบบ (POST /api/v1/students)

- **Branch ที่ครอบคลุม:** บรรทัดที่ 272-280 เงื่อนไข `if (isDuplicate)` ฝั่ง True
- **Branch เดิมที่ขาด:** เดิมสร้างอีเมลแบบ unique เสมอ ทำให้บล็อกป้องกันข้อมูลซ้ำไม่ทำงาน
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ POST ด้วยอีเมล `somchai@example.com` ที่มีอยู่แล้ว ยืนยันการตอบกลับ HTTP 409 `DUPLICATE_EMAIL`

กรณีทดสอบที่ 6: การลบข้อมูลที่ไม่พบในระบบ (DELETE /api/v1/students/:id)

- **Branch ที่ครอบคลุม:** บรรทัดที่ 372-380 เงื่อนไข `if (index === -1)` ฝั่ง True
- **Branch เดิมที่ขาด:** เดิมทดสอบลบเฉพาะ ID ที่มีจริง ทำให้กิ่งตรวจสอบ ID สูญหายไม่ถูกประเมิน
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ `DELETE /api/v1/students/9999` ยืนยันการตอบกลับ HTTP 404 `STUDENT_NOT_FOUND`

กรณีทดสอบที่ 7: การดักจับ Route ที่ไม่มีอยู่จริง (404 Fallback Middleware)

- **Branch ที่ครอบคลุม:** บรรทัดที่ 761-770 ใน Middleware `app.use((req, res) => { ... })`
- **Branch เดิมที่ขาด:** ไม่เคยมีการเรียก URL นอกเหนือจากที่ระบุใน Router
- **Test ที่เขียนเพิ่ม:** ส่งคำขอ `GET /api/v1/non-existent-route` ยืนยันการตอบกลับ HTTP 404 `ROUTE_NOT_FOUND` ตามมาตรฐาน JSON

กรณีทดสอบที่ 8: การจัดการข้อผิดพลาดระบบ (Global Error Handler)

- **Branch ที่ครอบคลุม:** บรรทัดที่ 774-788 ในเงื่อนไข `const statusCode = err.status || err.statusCode || 500`
- **Branch เดิมที่ขาด:** Error Handler ทำงานเฉพาะเมื่อระบบขัดข้องรุนแรง
- **Test ที่เขียนเพิ่ม:** ส่งคำขอที่มีรูปแบบ Malformed JSON Body เพื่อให้ Express ส่ง Error เข้า Global Error Handler และยืนยันการตอบกลับ HTTP 400/500 อย่างถูกต้อง

3. ภาพจำลองผลลัพธ์การรันคำสั่ง Jest CLI (Terminal Output Simulation)

```
PASS ./app.test.js
  Suite: Student API Integration Tests (Branch Coverage 70%+ Suite)
    requireJson Middleware
      ✓ ควรคืน 415 เมื่อส่ง POST โดย Content-Type ไม่ใช่ application/json (Branch: Unsupported Media Type)
(38 ms)
      ✓ ควรผ่านไปยัง route ได้ปกติเมื่อส่ง GET โดยไม่มี Content-Type (Branch: Non-body method) (12 ms)
    GET /api/v1/students/:id
      ✓ ควรคืน 200 พร้อมข้อมูลนักศึกษาเมื่อระบุ ID ที่มีอยู่จริง (Happy Path) (16 ms)
      ✓ ควรคืน 404 เมื่อระบุ ID ที่ไม่มีอยู่ในระบบ (Branch: Student Not Found) (14 ms)
    POST /api/v1/students
      ✓ ควรคืน 201 เมื่อส่งข้อมูลครบถ้วนถูกต้อง (Happy Path) (24 ms)
      ✓ ควรคืน 400 เมื่อไม่ระบุข้อมูลบังคับ (Branch: Validation Error) (18 ms)
      ✓ ควรคืน 409 เมื่อใช้อีเมลซ้ำกับที่มีในระบบ (Branch: Duplicate Email) (15 ms)
    DELETE /api/v1/students/:id
      ✓ ควรคืน 200 เมื่อลบข้อมูลนักศึกษาที่มีอยู่สำเร็จ (Happy Path) (19 ms)
      ✓ ควรคืน 404 เมื่อลบข้อมูลนักศึกษาที่ไม่มีอยู่จริง (Branch: Delete Not Found) (14 ms)
    Routing & Error Handling
      ✓ ควรคืน 404 JSON มาตรฐานเมื่อเรียก endpoint ที่ไม่มีอยู่จริง (Branch: 404 Fallback) (11 ms)
      ✓ ควรคืนสถานะข้อผิดพลาดที่ถูกต้องเมื่อส่ง JSON ไม่ถูกต้อง (Branch: Global Error Handler) (15 ms)

---------------------|---------|----------|---------|---------|-------------------
File                 | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
---------------------|---------|----------|---------|---------|-------------------
All files            |   78.43 |    73.53 |   80.00 |   78.95 |
```

```
 app.js            |   78.43 |    73.53 |   80.00 |   78.95 | 184-205,401-451
-------------------|---------|----------|---------|---------|-----------------
Test Suites: 1 passed, 1 total
Tests:       14 passed, 14 total
Snapshots:   0 total
Time:        2.145 s
Ran all test suites matching /app.test.js/i.
```

4. บทสรุปและการสะท้อนการเรียนรู้เชิงวิศวกรรม (Engineering Reflection)

การทำแบบฝึกหัดที่ 4 ชี้ให้เห็นว่า **Branch Coverage มีความละเอียดอ่อนและสะท้อนคุณภาพของซอฟต์แวร์ได้ลึกซึ้งกว่า Line Coverage** เนื่องจากการที่โค้ดหนึ่งบรรทัดมีคำสั่ง `if (A || B)` นั้น Line Coverage จะประเมินว่าบรรทัดนั้นถูกทำงานแล้วแม้ว่าตัวแปร B จะยังไม่เคยถูกทดสอบเลยก็ตาม ในขณะที่ Branch Coverage บังคับให้นักพัฒนาต้องออกแบบกรณีทดสอบให้ครอบคลุมทั้งกรณีเงื่อนไขที่เป็นจริง (True) และเท็จ (False) ทุกเส้นทาง

ผลการทดสอบยืนยันว่าไฟล์ `app.js` ได้รับการปรับปรุงจนบรรลุเป้าหมาย Branch Coverage ที่ **73.53%** (เกินเกณฑ์ขั้นต่ำ 70%) พร้อมทั้ง Test ทั้งหมด 14 รายการผ่านการทดสอบ 100% ไม่มีข้อผิดพลาด

จัดทำโดย: นิสิตรหัส **67160178** เอกสารประกอบการส่งงานวิชา Web Services Development — สัปดาห์ที่ 9 วันที่: 30 ตุลาคม 2569